



**Sistema de detección de intrusiones para redes IoT
mediante TinyML, aprendizaje federado jerárquico y
cifrado ligero ASCON-128 en una arquitectura
Edge–Fog–Cloud**

Santiago Alejandro Jaimes Puerto
Nicolas Casas Ibarra

Universidad de Los Andes
Departamento de Ingeniería de sistemas y computación
Bogotá, Colombia
2026

**Sistema de detección de intrusiones para redes IoT
mediante TinyML, aprendizaje federado jerárquico y
cifrado ligero ASCON-128 en una arquitectura
Edge-Fog-Cloud.**

Santiago Alejandro Jaimes Puerto
Nicolas Casas Ibarra

University de Los Andes
Presentado en cumplimiento parcial de los requisitos para el grado de:
System and Computer engineer

Director:
Prof. Carlos Andrés Lozano Garzón, PhD
Trabajo de grado defendido en:
Mayo 24, 2026 - 12:00 pm

Universidad de Los Andes
Departamento de Ingeniería de Sistemas y Computación
Bogotá, Colombia
2026

Agradecimientos

Quiero expresar mi más sincera gratitud a todas las personas que hicieron posible este logro.

A mi familia, quienes han sido mi ancla y mi inspiración en cada paso de este camino. Su amor incondicional y apoyo constante me han dado la fortaleza para superar cualquier desafío.

Al profesor Paulo de Oliveria, mi asesor, quien no solo compartió generosamente su conocimiento y experiencia, sino que también fue un guía incansable durante este proyecto. Su paciencia, dedicación y constante disposición para resolver dudas fueron esenciales para enfrentar los retos que surgieron. Sus valiosos comentarios y orientaciones me permitieron aprender y crecer, tanto a nivel académico como personal. Este proyecto no habría sido lo mismo sin su acompañamiento e inspiración.

A la Universidad de los Andes, mi alma máter, por ser un espacio de excelencia académica que me permitió desarrollar mis habilidades y consolidar mi formación profesional. A sus profesores, administrativos y recursos de investigación, les agradezco por fomentar un ambiente de aprendizaje integral, colaboración y crecimiento personal. Haber formado parte de esta comunidad académica ha sido un privilegio invaluable que marcará para siempre mi trayectoria.

Y, con un cariño especial, a mis amigos. Gracias por convertir este recorrido en una aventura inolvidable. A Sebastián Moya, por su apoyo clave y su disposición.

A todos ustedes, mi eterno agradecimiento. Este logro también es suyo.

Resumen

Este documento presenta el diseño, implementación y evaluación de un sistema de detección de intrusiones (IDS) para redes IoT/MQTT desplegado en una arquitectura jerárquica Edge-Fog-Cloud. El sistema combina modelos livianos de *machine learning* ejecutados en microcontroladores ESP32-S3, aprendizaje federado jerárquico (HFL) coordinado desde gateways Raspberry Pi 4 y un servidor PC, y cifrado ligero **ASCON-128** para proteger el ciclo de aprendizaje. La investigación evalúa cuatro dimensiones: viabilidad de TinyML en el borde, reducción de transferencia de datos mediante HFL, overhead criptográfico de ASCON y comparación entre variantes de arquitectura y modelo.

Se diseñó un perceptrón multicapa (MLP) compacto de 1 139 parámetros (≈ 4.5 KB) con arquitectura $13 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 3$, capaz de clasificar flujos de red en tres categorías (`normal`, `mqtt_bruteforce`, `scan_A`) utilizando 13 características extraídas de flujos. El sistema federado comparte únicamente las dos últimas capas del modelo (163 parámetros) y fue evaluado en las ramas `hfl_v7-RN`, `hfl_v7-no-ascon` y `hfl_v7-CNN`, incluyendo modo estándar y modo FOG.

Como línea base, se entrenaron modelos clásicos centralizados, alcanzando hasta 99.94 % de accuracy con Random Forest, aunque con requerimientos de memoria incompatibles con microcontroladores. El notebook general de entrenamiento comparó además MLP/RN, CNN-1D, Residual-MLP y Tiny Transformer; el MLP/RN obtuvo el mejor balance entre precisión offline ($\approx 90.6\%$), tamaño e implementación en ESP32. La cuantificación del overhead de ASCON-128 mostró costo temporal bajo frente a la duración de las rondas federadas y un incremento moderado del tamaño de los mensajes, sin degradación observable de convergencia.

Los resultados validan que la combinación de TinyML, aprendizaje federado y cifrado ligero constituye una solución viable y segura para detección de intrusiones en el borde de redes IoT, ofreciendo un compromiso efectivo entre precisión, privacidad, seguridad y eficiencia computacional.

Keywords: Detección de Intrusiones, IoT, TinyML, Aprendizaje Federado Jerárquico, ASCON-128, Criptografía Ligera, Edge-Fog-Cloud, ESP32-S3, Raspberry Pi, CNN-1D, MLP, Seguridad de Redes.

Tabla de Contenido

List of Figures	XI
List of Tables	XII
1. Introducción	1
1.1. Contexto	1
1.2. Problemática	1
1.3. Justificación	2
1.4. Objetivos	3
1.4.1. Objetivo General	3
1.4.2. Objetivos Específicos	3
1.4.3. Preguntas de investigación	3
2. Marco Teórico	4
2.1. Marco Conceptual	4
2.1.1. IDS en redes IoT/IIoT	4
2.1.2. Representación por flujos	4
2.1.3. TinyML y modelos compactos	4
2.1.4. Aprendizaje federado jerárquico	4
2.1.5. Cifrado ligero y ASCON-128	5
2.1.6. Agregación FOG	5
2.2. Estado del Arte	5
2.3. Síntesis de la Brecha	6
3. Metodología	7
3.1. Arquitectura del Sistema	7
3.1.1. Capa Edge: Nodos ESP32-S3	7
3.1.2. Capa Fog: Gateway Raspberry Pi 4	8
3.1.3. Capa Cloud: Servidor PC	8
3.1.4. Modos de operación	8
3.2. Conjunto de Datos	9
3.3. Modelo de Aprendizaje: MLP Multiclase	10
3.4. Modelos livianos evaluados	10
3.5. Protocolo de Aprendizaje Federado Jerárquico	11

3.6.	Integración de Cifrado Ligero ASCON-128	12
3.6.1.	Parámetros Criptográficos	12
3.6.2.	Canales Cifrados	12
3.6.3.	Métricas de Overhead	13
3.7.	Variantes implementadas en <code>hfl_v7</code>	13
3.8.	Modelos Clásicos de ML como Línea Base	13
3.9.	Protocolo Experimental	14
3.9.1.	Escenario 1: Sistema HFL Completo con ASCON	14
3.9.2.	Escenario 2: Sistema HFL sin ASCON (Baseline)	15
3.9.3.	Escenario 3: Sistema HFL con CNN-1D	15
3.9.4.	Escenario 4: Estrategia FOG	15
3.9.5.	Escenario 5: Modelos Clásicos Centralizados	15
3.9.6.	Métricas de Evaluación	15
4.	Resultados	16
4.1.	Desempeño de Modelos Clásicos (Línea Base Centralizada)	16
4.1.1.	Viabilidad de Despliegue en ESP32	17
4.2.	Comparación de Modelos Livianos	17
4.3.	Aprendizaje Federado Jerárquico: Convergencia y Precisión	18
4.3.1.	Convergencia del Modelo Federado	18
4.3.2.	Evolución temporal de accuracy, loss y pesos	19
4.3.3.	Comparación: Federado vs. Centralizado	20
4.3.4.	Comparación entre variantes <code>hfl_v7</code>	21
4.4.	Overhead del Cifrado ASCON-128	21
4.4.1.	Overhead Temporal	22
4.4.2.	Overhead de Comunicación	22
4.4.3.	Resumen: Impacto de ASCON en el Sistema HFL	23
4.5.	Análisis de las Preguntas de Investigación	23
4.5.1.	PI-1: ¿Puede TinyML detectar ataques de red en dispositivos IoT con precisión operativa?	23
4.5.2.	PI-2: ¿Reduce el aprendizaje federado el intercambio de datos sensibles sin afectar la precisión del IDS?	24
4.5.3.	PI-3: ¿Cuál es el overhead del cifrado ASCON en la comunicación de modelos federados?	24
4.5.4.	PI-4: ¿Qué aporta comparar MLP/RN, CNN-1D, baseline sin ASCON y FOG?	24
5.	Conclusiones y Trabajos Futuros	26
5.1.	Conclusiones	26
5.1.1.	Sobre la viabilidad de TinyML para IDS en IoT	26

5.1.2.	Sobre el aprendizaje federado y la privacidad	26
5.1.3.	Sobre el overhead del cifrado ASCON-128	27
5.1.4.	Sobre la arquitectura integrada	27
5.2.	Trabajos Futuros	28
Bibliografía		29
A. Anexos		32
A.1.	Anexo A: Arquitectura del Sistema HFL	32
A.2.	Anexo B: Archivos del Sistema	32
A.3.	Anexo C: Repositorio del Proyecto	33

Lista of Figuras

4-1.	Matriz comparativa 2×2 de accuracy y loss global para variantes HFL v7: RN con ASCON, RN sin ASCON, CNN-1D y CNN-1D con FOG.	19
4-2.	Matriz comparativa 2×2 de magnitud promedio de pesos globales para va- riantes HFL v7: RN con ASCON, RN sin ASCON, CNN-1D y CNN-1D con FOG.	20

Lista de Tablas

2-1.	Relación entre literatura y decisiones de diseño.	6
3-1.	Composición del dataset de flujos de red.	9
3-2.	Vector de 13 features extraídas por flujo.	9
3-3.	Distribución de parámetros del modelo MLP.	10
3-4.	Modelos livianos entrenados y artefactos exportados.	10
3-5.	Canales de comunicación protegidos con ASCON-128.	12
3-6.	Variantes de arquitectura evaluadas.	13
4-1.	Desempeño de modelos clásicos de ML (entrenamiento centralizado).	16
4-2.	Estimación de huella de memoria para despliegue en ESP32.	17
4-3.	Comparación offline de modelos livianos sobre las 13 features.	17
4-4.	Métricas de convergencia del sistema HFL.	18
4-5.	Comparación de precisión: entrenamiento federado vs. centralizado.	21
4-6.	Resumen comparativo de variantes HFL evaluadas.	21
4-7.	Tiempos de cifrado/descifrado ASCON-128 en el servidor central (PC).	22
4-8.	Overhead de tamaño por cifrado ASCON-128 + Base64.	22
4-9.	Resumen del overhead total de ASCON-128 por ronda federada.	23
A-1.	Archivos principales del sistema HFL v7.	32

1. Introducción

1.1. Contexto

La adopción de redes *Internet of Things* (IoT) e *Industrial IoT* (IIoT) ha aumentado la superficie de ataque de hogares, laboratorios, plantas industriales y sistemas de monitoreo conectados. Estos entornos combinan dispositivos heterogéneos, protocolos livianos como MQTT, conectividad intermitente y capacidades limitadas de cómputo, lo que dificulta desplegar mecanismos de detección convencionales basados en procesamiento centralizado. Bajo marcos de gestión de riesgo como el *NIST Cybersecurity Framework*, la detección temprana y el monitoreo continuo son capacidades fundamentales para reducir el tiempo de respuesta ante incidentes [1].

Los sistemas de detección de intrusiones de red (NIDS) basados en aprendizaje automático han mostrado altos niveles de precisión en condiciones de laboratorio. Sin embargo, muchos de estos resultados se obtienen bajo supuestos que no siempre se cumplen en IoT: centralización de datos, disponibilidad de servidores con alta memoria, ausencia de cifrado en las comunicaciones internas del sistema y modelos demasiado grandes para ejecutarse en microcontroladores. En consecuencia, el reto no consiste únicamente en maximizar *accuracy*, sino en validar una arquitectura completa que pueda operar sobre hardware restringido, actualizarse de manera distribuida y proteger los artefactos del aprendizaje.

Esta tesis aborda ese reto mediante una arquitectura Edge-Fog-Cloud para detección de intrusiones en tráfico MQTT. La capa Edge se implementa con nodos ESP32-S3 que ejecutan inferencia TinyML y transmiten características de flujo; la capa Fog se implementa sobre Raspberry Pi 4 como gateway MQTT, entrenador local y agregador intermedio; y la capa Cloud se implementa en un PC con FastAPI para coordinar la agregación global. El sistema se evaluó en varias configuraciones de la versión `hf1_v7`: red neuronal MLP con ASCON-128, red neuronal MLP sin ASCON como línea base, variante CNN-1D y estrategia FOG con agregación entre gateways.

1.2. Problemática

La primera brecha identificada es la **viabilidad de despliegue**. Modelos centralizados como Random Forest o SVM-RBF pueden alcanzar precisiones cercanas al 100 %, pero sus requerimientos de memoria son incompatibles con microcontroladores. En contraste, un IDS

IoT real necesita inferencia frecuente, bajo consumo de memoria y comunicación limitada.

La segunda brecha corresponde a la **privacidad y distribución de datos**. Centralizar capturas PCAP o flujos completos desde múltiples redes introduce costos de transferencia y riesgos de exposición. El aprendizaje federado reduce este problema al compartir pesos o gradientes en lugar de datos crudos, pero en IoT introduce retos adicionales: datos *non-IID*, rondas incompletas, heterogeneidad de hardware y sobrecarga de comunicación [2, 3].

La tercera brecha es la **seguridad del propio ciclo federado**. Aunque FL evita mover datos crudos, los pesos del modelo, métricas locales y mensajes de control siguen siendo activos sensibles. Por tanto, las comunicaciones entre ESP32, Raspberry Pi y PC deben proteger confidencialidad e integridad con mecanismos compatibles con dispositivos restringidos. ASCON-128, estandarizado por NIST como criptografía ligera, es una opción pertinente para este escenario [4].

Finalmente, el sistema debe ser evaluado como arquitectura, no solo como modelo. Por eso se comparan cuatro dimensiones: desempeño del modelo TinyML, costo del aprendizaje federado, overhead de ASCON-128 y efecto de variar el modelo o la topología mediante las ramas `hfl_v7-RN`, `hfl_v7-no-ascon` y `hfl_v7-CNN`.

1.3. Justificación

La solución propuesta aporta una validación práctica de un IDS federado para IoT sobre hardware real. A diferencia de un entrenamiento centralizado, el sistema opera con un modelo compacto de 13 características de flujo y tres clases de tráfico: `normal`, `mqtt_bruteforce` y `scan_A`. Este alcance permite instrumentar el ciclo completo de detección, entrenamiento local, agregación global y redistribución del modelo sin depender de infraestructura de alto costo en el borde.

El proyecto también justifica la comparación entre variantes. La rama con ASCON-128 permite medir el costo de proteger todos los canales; la rama sin ASCON sirve como línea base para aislar el overhead criptográfico; la variante CNN-1D evalúa si una arquitectura más expresiva mejora el desempeño sin romper la viabilidad edge; y el modo FOG introduce una agregación intermedia entre Raspberry Pi antes del servidor central.

El documento está estructurado de la siguiente manera: el Capítulo 1 presenta el problema y los objetivos; el Capítulo 2 resume el marco conceptual y el estado del arte; el Capítulo 3 describe la arquitectura y el protocolo experimental; el Capítulo 4 presenta resultados; el Capítulo 5 concluye y plantea trabajos futuros; y los anexos resumen archivos y artefactos del sistema.

1.4. Objetivos

1.4.1. Objetivo General

Diseñar, implementar y evaluar un sistema de detección de intrusiones para tráfico IoT/MQTT basado en TinyML, aprendizaje federado jerárquico y cifrado ligero ASCON-128, desplegado sobre una arquitectura Edge-Fog-Cloud compuesta por ESP32-S3, Raspberry Pi 4 y un servidor PC.

1.4.2. Objetivos Específicos

1. Construir un *pipeline* de datos basado en flujos de red unidireccionales, seleccionando 13 características computables en dispositivos de borde y definiendo un problema triclase para tráfico `normal`, `mqtt_bruteforce` y `scan_A`.
2. Entrenar y comparar modelos compactos para IDS IoT, incluyendo MLP, CNN-1D, Residual-MLP y Tiny Transformer, exportando pesos y parámetros de normalización para ejecución en ESP32-S3.
3. Implementar una arquitectura HFL bidireccional donde los gateways Raspberry Pi entrenan localmente, envían pesos al servidor central, reciben el modelo global y lo redistribuyen a los nodos ESP32.
4. Evaluar el impacto del cifrado ASCON-128 frente a una línea base sin cifrado, midiendo tiempo de cifrado/descifrado, tamaño de mensajes y efecto sobre la convergencia del modelo.
5. Analizar el efecto de variar la arquitectura del modelo y la topología de agregación mediante las ramas `hfl_v7-RN`, `hfl_v7-no-ascon`, `hfl_v7-CNN` y el modo FOG.

1.4.3. Preguntas de investigación

1. ¿Puede un modelo TinyML ejecutado en ESP32-S3 detectar tráfico MQTT malicioso con precisión operativamente viable?
2. ¿Qué reducción de transferencia de datos se obtiene al usar HFL en lugar de centralizar los flujos de red?
3. ¿Cuál es el costo temporal y de comunicación de proteger el ciclo federado con ASCON-128?
4. ¿Cómo cambian los resultados al comparar MLP, CNN-1D, baseline sin ASCON y agregación FOG?

2. Marco Teórico

2.1. Marco Conceptual

2.1.1. IDS en redes IoT/IIoT

Un sistema de detección de intrusiones de red (NIDS) monitorea tráfico y eventos para identificar comportamientos maliciosos o anómalos. En IoT/IIoT, esta tarea es más exigente que en redes tradicionales porque los dispositivos tienen recursos limitados, usan protocolos livianos, presentan patrones de tráfico heterogéneos y suelen estar distribuidos en dominios administrativos distintos. Por ello, la detección debe evaluarse no solo por *accuracy*, sino también por latencia, memoria, comunicación y estabilidad operacional [5, 6].

2.1.2. Representación por flujos

El tráfico de red puede representarse como paquetes, capturas PCAP, logs o flujos. Esta tesis utiliza flujos unidireccionales porque son compactos, tabulares y computables en el borde. Cada flujo se transforma en 13 características numéricas relacionadas con conteo de paquetes, tiempos entre paquetes, tamaños de paquetes, bytes totales y banderas TCP. Esta representación sacrifica parte del detalle secuencial de un PCAP completo, pero permite inferencia TinyML en ESP32-S3 y reduce el volumen de comunicación.

2.1.3. TinyML y modelos compactos

TinyML agrupa técnicas para ejecutar modelos de aprendizaje automático en hardware restringido. En el contexto de esta tesis, el criterio de diseño es que el modelo pueda ejecutarse directamente en microcontroladores ESP32-S3 sin depender de aceleradores externos ni transferir los datos crudos al servidor. Los modelos evaluados incluyen MLP, CNN-1D, Residual-MLP y Tiny Transformer; sin embargo, el despliegue principal utiliza un MLP compacto por su equilibrio entre tamaño, precisión y simplicidad de inferencia.

2.1.4. Aprendizaje federado jerárquico

El aprendizaje federado (FL) permite entrenar modelos de forma distribuida compartiendo actualizaciones en lugar de datos crudos. En escenarios IoT, una arquitectura jerárquica (HFL) resulta natural porque los nodos edge pueden comunicarse con gateways cercanos y

estos, a su vez, con un coordinador central. Este diseño reduce exposición de datos, organiza la comunicación por capas y permite agregar actualizaciones localmente antes de enviarlas al servidor global [7, 2].

La literatura advierte que FL en IoT enfrenta datos *non-IID*, conectividad irregular y heterogeneidad de hardware [3, 8]. En esta tesis, esas condiciones se reproducen parcialmente mediante nodos con distribuciones distintas: un nodo genera tráfico normal y otro genera una mezcla de tráfico normal, fuerza bruta MQTT y escaneo.

2.1.5. Cifrado ligero y ASCON-128

Aunque FL reduce la centralización de datos, los pesos del modelo y mensajes de control siguen siendo sensibles. Un atacante que intercepte o modifique actualizaciones podría inferir información, degradar el modelo o introducir comportamiento malicioso. Por esta razón, la arquitectura integra ASCON-128 como cifrado autenticado ligero para proteger confidencialidad e integridad en los canales MQTT y HTTP del ciclo federado.

ASCON fue seleccionado por NIST para criptografía ligera en dispositivos restringidos, y su diseño resulta apropiado para mensajes pequeños como features, pesos parciales y modelos globales [4, 9]. En esta tesis se mide explícitamente su overhead frente a una versión funcionalmente equivalente sin cifrado.

2.1.6. Agregación FOG

El modo FOG extiende la arquitectura estándar con comunicación entre gateways Raspberry Pi. Un gateway líder recibe actualizaciones de gateways pares mediante MQTT, realiza un FedAvg intermedio y envía al servidor central una actualización preagregada. Este esquema aproxima escenarios reales con múltiples zonas o sedes, donde una capa fog reduce tráfico hacia la nube y permite coordinación local antes de la agregación global.

2.2. Estado del Arte

La literatura revisada se agrupa en tres líneas. La primera aborda IDS para IoT/IIoT y destaca que los modelos de alta precisión no siempre son desplegados en el borde. Trabajos recientes sobre TinyML e IDS enfatizan que la memoria, la latencia y el costo energético deben reportarse junto con las métricas de clasificación [5, 10, 6].

La segunda línea corresponde al aprendizaje federado aplicado a seguridad IoT. FL permite entrenar modelos sin centralizar datos, pero introduce retos de comunicación, datos *non-IID* y participación parcial. Estudios recientes sobre FL para IDS muestran que la precisión puede degradarse frente al entrenamiento centralizado, pero que la reducción de datos compartidos puede justificar el intercambio en escenarios con restricciones de privacidad [2, 3, 11].

La tercera línea es criptografía ligera. ASCON y otros esquemas de bajo costo buscan proteger dispositivos restringidos sin introducir overhead prohibitivo. Para un IDS federado, esta línea es especialmente relevante porque las actualizaciones de modelo son parte del activo de seguridad y deben protegerse durante el tránsito [4, 12].

2.3. Síntesis de la Brecha

Los trabajos existentes suelen cubrir por separado TinyML, FL o cifrado ligero. La brecha abordada en esta tesis es la integración y medición de un ciclo completo Edge–Fog–Cloud sobre hardware real, con inferencia en ESP32-S3, entrenamiento local en Raspberry Pi, agregación global en PC, protección ASCON-128, línea base sin cifrado, variante CNN y modo FOG. Esta evaluación conjunta permite discutir el sistema como una arquitectura operativa y no solo como un clasificador aislado.

Table 2-1.: Relación entre literatura y decisiones de diseño.

Línea	Referencias	Uso en la tesis
TinyML/IDS IoT	[5, 6, 10]	Selección de modelos compactos y reporte de viabilidad en ESP32-S3.
FL/HFL	[7, 2, 3]	Diseño de FedAvg jerárquico y discusión de datos <i>non-IID</i> .
Criptografía ligera	[4, 12]	Integración de ASCON-128 y comparación contra baseline sin cifrado.

3. Metodología

La presente investigación sigue un enfoque experimental y cuantitativo, estructurado en seis fases: (i) construcción del pipeline de datos, (ii) entrenamiento y comparación offline de modelos livianos, (iii) implementación de la arquitectura jerárquica de aprendizaje federado, (iv) integración de cifrado ligero ASCON-128, (v) construcción de una línea base sin cifrado y (vi) evaluación de variantes con MLP/RN, CNN-1D y estrategia FOG sobre hardware real.

3.1. Arquitectura del Sistema

El sistema propuesto se organiza en tres capas jerárquicas que reflejan la topología Edge–Fog–Cloud, cada una con roles diferenciados de procesamiento, entrenamiento y coordinación.

3.1.1. Capa Edge: Nodos ESP32-S3

Los nodos de borde se implementan sobre microcontroladores **ESP32-S3** con conectividad Wi-Fi nativa. Cada nodo cumple tres funciones:

1. **Generación y captura de tráfico:** simula flujos de red MQTT con patrones estadísticos calibrados a partir de los datasets reales (Sección 3.2).
2. **Extracción de características:** computa en tiempo real un vector de 13 features por flujo (Tabla 3-2).
3. **Inferencia local (TinyML):** ejecuta un modelo compacto directamente en el microcontrolador, clasificando cada flujo en tres categorías: `normal`, `mqtt_bruteforce` y `scan_A`.

Se despliegan dos nodos diferenciados: un nodo *normal* que genera exclusivamente tráfico benigno, y un nodo *simulado* que genera una mezcla de 40 % normal, 30 % bruteforce y 30 % scan_A, reproduciendo las distribuciones estadísticas reales de los datasets. En la rama RN se usa un MLP; en la rama CNN se usa una CNN-1D con capas convolucionales fijas y capas densas federadas.

3.1.2. Capa Fog: Gateway Raspberry Pi 4

La Raspberry Pi 4 actúa como *gateway* Fog y ejecuta `gateway_hfl.py` o `gateway_hfl_fog.py`, según el escenario experimental. Sus responsabilidades incluyen:

- Actuar como **broker MQTT** (Mosquitto) para recibir las features de los ESP32.
- Mantener un **buffer de entrenamiento** de $N_s = 40$ muestras etiquetadas mediante una función heurística basada en las distribuciones reales del dataset.
- **Entrenar localmente** un modelo Keras/TensorFlow idéntico al del ESP32 (5 épocas, Adam con $\eta = 0,005$, batch=8).
- **Transmitir los pesos** de las capas federadas al servidor central vía HTTP.
- **Distribuir el modelo global** recibido del servidor a los ESP32 vía MQTT.
- En modo FOG, **agregar pesos entre gateways Raspberry Pi** antes de enviar una actualización preagregada al PC.

3.1.3. Capa Cloud: Servidor PC

Un PC convencional ejecuta `server_hfl.py` o `server_hfl_fog.py` (FastAPI + Uvicorn) y coordina el aprendizaje federado global:

- Recibe los pesos locales de K gateways (en esta implementación, $K = 2$).
- Aplica el algoritmo **FedAvg** (Sección 3.5) ponderado por número de muestras.
- Distribuye el modelo global actualizado a todos los gateways.
- Proporciona un **dashboard analítico** en tiempo real con gráficas de accuracy y loss por ronda.

3.1.4. Modos de operación

La versión `hfl_v7` fue implementada en dos modos:

- **Modo estándar:** cada Raspberry Pi actúa como gateway independiente. El servidor PC espera actualizaciones de $K = 2$ gateways y aplica FedAvg global.
- **Modo FOG:** una Raspberry Pi líder recibe pesos de una Raspberry Pi par mediante MQTT, calcula FedAvg local de la capa fog y envía al PC una actualización preagregada. El modelo global retorna al líder y se redistribuye a los peers y a los ESP32.

3.2. Conjunto de Datos

Se utilizan tres datasets públicos de flujos de red IoT, cada uno representando una clase de tráfico. Estos provienen de capturas PCAP procesadas y convertidas a representaciones de flujo unidireccional (*uniflow*):

Table 3-1.: Composición del dataset de flujos de red.

Dataset	Flujos	Etiqueta	Protocolo principal
<code>uniflow_normal.csv</code>	171 837	0 (<code>normal</code>)	MQTT, TCP
<code>uniflow_mqtt_bruteforce.csv</code>	33 080	1 (<code>mqttd_bruteforce</code>)	MQTT (login)
<code>uniflow_scan_A.csv</code>	51 359	2 (<code>scan_A</code>)	TCP SYN
Total	256 276	3 clases	—

Cada flujo se representa mediante un vector de $d = 13$ características numéricas, seleccionadas para ser computables en tiempo real por un microcontrolador:

Table 3-2.: Vector de 13 features extraídas por flujo.

Índice	Feature	Descripción
0	<code>num_pkts</code>	Número de paquetes del flujo
1	<code>mean_iat</code>	Inter-arrival time promedio (s)
2	<code>std_iat</code>	Desviación estándar del IAT
3	<code>min_iat</code>	IAT mínimo
4	<code>max_iat</code>	IAT máximo
5	<code>mean_pkt_len</code>	Longitud promedio de paquete (bytes)
6	<code>num_bytes</code>	Total de bytes del flujo
7	<code>num_psh_flags</code>	Conteo de flags PSH
8	<code>num_rst_flags</code>	Conteo de flags RST
9	<code>num_urg_flags</code>	Conteo de flags URG
10	<code>std_pkt_len</code>	Desviación estándar de longitud de paquete
11	<code>min_pkt_len</code>	Longitud mínima de paquete
12	<code>max_pkt_len</code>	Longitud máxima de paquete

Para el preprocesamiento, se aplica un `StandardScaler` que normaliza cada feature a media cero y varianza unitaria. Los parámetros del scaler (μ_i, σ_i) se exportan como constantes embebidas en el firmware del ESP32, permitiendo la normalización en el dispositivo sin dependencia de bibliotecas externas.

3.3. Modelo de Aprendizaje: MLP Multiclase

Se diseñó un perceptrón multicapa (MLP) compacto orientado a la ejecución en microcontroladores con restricciones de memoria:

$$\text{MLP} : \mathbb{R}^{13} \xrightarrow{W_1, b_1} \mathbb{R}^{32} \xrightarrow{W_2, b_2} \mathbb{R}^{16} \xrightarrow{W_3, b_3} \mathbb{R}^8 \xrightarrow{W_4, b_4} \mathbb{R}^3 \quad (3-1)$$

Las primeras tres capas utilizan activación ReLU y la capa de salida emplea Softmax para obtener probabilidades sobre las 3 clases. La Tabla **3-3** detalla la distribución de parámetros:

Table 3-3.: Distribución de parámetros del modelo MLP.

Capa	Entrada	Salida	Parámetros
Dense 1 (ReLU)	13	32	$13 \times 32 + 32 = 448$
Dense 2 (ReLU)	32	16	$32 \times 16 + 16 = 528$
Dense 3 (ReLU)	16	8	$16 \times 8 + 8 = 136$
Dense 4 (Softmax)	8	3	$8 \times 3 + 3 = 27$
Total			1 139

Con 1 139 parámetros en punto flotante de 32 bits, el modelo completo ocupa $\approx 4,5$ KB de memoria, lo que permite su ejecución directa en la SRAM del ESP32-S3 sin necesidad de cuantización.

En el esquema de aprendizaje federado, solo las **dos últimas capas** (W_3, b_3, W_4, b_4 , 163 parámetros) son intercambiadas entre nodos, reduciendo el costo de comunicación a ≈ 652 bytes por actualización (sin serialización JSON).

3.4. Modelos livianos evaluados

Antes del despliegue federado se ejecutó un notebook general de entrenamiento con cuatro familias de modelos sobre las mismas 13 características y las mismas tres clases. El objetivo fue seleccionar una arquitectura viable para ESP32-S3 y, al mismo tiempo, documentar alternativas para comparación experimental:

Table 3-4.: Modelos livianos entrenados y artefactos exportados.

Modelo	Arquitectura	Artefactos exportados
MLP/RN	$13 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 3$	<code>model_weights.h</code> , <code>ids_3clas</code>
CNN-1D	$\text{Conv1D}(32) \rightarrow \text{Conv1D}(16) \rightarrow \text{GAP} \rightarrow \text{Dense8} \rightarrow \text{Dense3}$	Pesos Conv fusionados + capa
Residual-MLP	MLP compacto con conexión residual	Pesos, scaler y modelo Keras
Tiny Transformer	Auto-atención tabular liviana	Pesos y modelo Keras para an

El despliegue principal selecciona el MLP/RN porque ofrece el mejor balance entre precisión offline, tamaño y simplicidad de implementación en C++ sobre ESP32-S3. La variante CNN-1D se implementa como rama experimental: las capas convolucionales permanecen fijas en el edge y solo se federan las capas densas `dense_1` y `dense_out`, equivalentes funcionalmente a las capas federadas del MLP.

3.5. Protocolo de Aprendizaje Federado Jerárquico

Se implementa un esquema de aprendizaje federado jerárquico (HFL) con flujo bidireccional:

1. **Nivel Edge (ESP32 → RPi):** Los nodos ESP32 envían vectores de features al gateway cada 5 segundos vía MQTT (`f1/features`). El gateway acumula $N_s = 40$ muestras etiquetadas heurísticamente.
2. **Nivel Fog (RPi → PC o RPi → RPi líder):** Tras llenar el buffer, el gateway entrena localmente durante $E = 5$ épocas y envía las capas federadas junto con la accuracy local y el número de muestras.
3. **Nivel Cloud (Agregación FedAvg):** El servidor espera recibir actualizaciones de $K = 2$ gateways y aplica FedAvg ponderado por número de muestras:

$$W_{\text{global}}^{(t+1)} = \frac{\sum_{k=1}^K n_k \cdot W_k^{(t)}}{\sum_{k=1}^K n_k} \quad (3-2)$$

donde n_k es el número de muestras usadas para el entrenamiento local del gateway k y $W_k^{(t)}$ sus pesos locales en la ronda t .

El modelo global resultante es distribuido de vuelta a los gateways (HTTP POST a `/deploy-model`), quienes a su vez lo publican a los ESP32 por MQTT (`f1/global_model`). En la rama sin ASCON se usan topics y endpoints con sufijo `_plain`, por ejemplo `f1/features_plain`, `f1/global_model_plain`, `/aggregate-from-gateway-plain` y `/deploy-model-plain`. De esta forma, el flujo bidireccional completo es:

$$\text{ESP32} \xrightarrow{\text{features (MQTT)}} \text{RPi} \xrightarrow{\text{pesos (HTTP)}} \text{PC} \xrightarrow{\text{modelo global (HTTP)}} \text{RPi} \xrightarrow{\text{modelo global (MQTT)}} \text{ESP32}$$

En modo FOG, el flujo agrega dos canales adicionales: `fog/weights` para que los peers envíen pesos al líder y `fog/global_model` para que el líder redistribuya el modelo global a los peers.

El etiquetado en el gateway se realiza mediante una **función heurística** calibrada con las distribuciones estadísticas reales de los datasets:

- Si `num_pkts` ≥ 50 y `num_psh` ≥ 10 : etiqueta = 1 (bruteforce).

- Si $\text{num_pkts} \leq 5$ y $\text{mean_pkt_len} \leq 50$ y $\text{num_psh} \leq 1$: etiqueta = 2 (scan_A).
- Si $\text{num_pkts} \leq 30$ y $\text{mean_pkt_len} \geq 50$: etiqueta = 0 (normal).
- En otro caso, la muestra se descarta (-1).

3.6. Integración de Cifrado Ligero ASCON-128

Para proteger los artefactos de aprendizaje federado en tránsito, se integra **ASCON-128** [4], el estándar de criptografía ligera seleccionado por NIST para dispositivos restringidos. ASCON provee cifrado autenticado con datos asociados (AEAD), garantizando simultáneamente *confidencialidad* e *integridad* de los mensajes.

3.6.1. Parámetros Criptográficos

- **Clave simétrica:** 128 bits (16 bytes), pre-compartida entre todos los dispositivos.
- **Nonce:** 128 bits, generado a partir de timestamp truncado a 32 bits y un contador monotónico.
- **Tag de autenticación:** 128 bits (16 bytes).
- **Tasa (rate):** 64 bits (8 bytes por bloque).
- **Permutación:** p^{12} para inicialización/finalización, p^6 para procesamiento de bloques.

3.6.2. Canales Cifrados

Se cifran los **cuatro canales de comunicación** del sistema:

Table 3-5.: Canales de comunicación protegidos con ASCON-128.

Canal	Protocolo	Contenido	Implementación
ESP32 $\rightarrow$ RPi	MQTT	Features (13 floats)	C++ (<code>ascon128.h</code>)
RPi $\rightarrow$ PC	HTTP	Pesos locales + métricas	Python (<code>ascon128.py</code>)
PC $\rightarrow$ RPi	HTTP	Modelo global	Python (<code>ascon128.py</code>)
RPi $\rightarrow$ ESP32	MQTT	Modelo global	Python $\rightarrow$ C++

El payload cifrado se encapsula en un *envelope* JSON con tres campos codificados en Base64:

```
{"ct": "<ciphertext_b64>", "tag": "<tag_b64>", "nonce": "<nonce_b64>"}
```


3.6.3. Métricas de Overhead

Para cuantificar el overhead de ASCON, se instrumentan todas las operaciones de cifrado y descifrado con temporizadores de alta resolución (`micros()` en ESP32, `time.perf_counter()` en Python) y se registran automáticamente en archivos CSV mediante el módulo `ascon_metrics.py`, capturando:

- Tiempo de cifrado/descifrado (ms).
- Tamaño del plaintext y del envelope cifrado (bytes).
- Overhead de comunicación (bytes adicionales y porcentaje).
- Ronda de FL asociada.

Adicionalmente, se mantiene una **rama sin cifrado** (`hfl_v7-no-ascon`) del repositorio con idéntica funcionalidad pero sin ASCON, permitiendo la comparación directa de tiempos de comunicación como línea base. En esta rama los mensajes se serializan como JSON plano y se registran mediante `plain_metrics.py`.

3.7. Variantes implementadas en hfl_v7

Table 3-6.: Variantes de arquitectura evaluadas.

Carpeta	Modelo	Seguridad	Topología
<code>hfl_v7-RN</code>	MLP/RN	ASCON-128	Estándar y FOG
<code>hfl_v7-no-ascon</code>	MLP/RN	JSON plano	Estándar y FOG
<code>hfl_v7-CNN</code>	CNN-1D	ASCON-128	Estándar y FOG

Estas variantes permiten aislar tres efectos: el impacto del cifrado, el impacto de cambiar el modelo y el impacto de introducir una agregación fog intermedia.

3.8. Modelos Clásicos de ML como Línea Base

Para responder a la pregunta de si TinyML puede alcanzar precisión comparable a modelos tradicionales, se entrenan y optimizan cuatro algoritmos de ML supervisado sobre los mismos datasets:

1. **Decision Tree:** Optimizado con `GridSearchCV` sobre `max_depth`, `min_samples_split`, `min_samples_leaf` y `criterion`. Incluye `class_weight='balanced'` para manejar el desbalance de clases.

2. **Random Forest:** Optimizado con `RandomizedSearchCV` ($n = 80$ iteraciones) sobre `n_estimators`, `max_depth`, `max_features` y variantes de `class_weight`.
3. **Logistic Regression:** Evaluada con penalizaciones L1, L2 y ElasticNet, variando el parámetro de regularización C .
4. **Support Vector Machine (SVM):** Se comparan kernel lineal y RBF. Para el kernel RBF, se aplica submuestreo estratificado (máximo 25 000 muestras por clase) para evitar tiempos de entrenamiento prohibitivos.

Todos los modelos se evalúan con validación cruzada estratificada ($k = 5$) usando F1-Score macro como métrica de selección. Para estimar la viabilidad de despliegue en ESP32, se utiliza la biblioteca `m2cgen` para exportar los modelos entrenados a código C puro, y se mide:

- Tamaño estimado en memoria del modelo exportado.
- Tiempo de inferencia promedio (100 iteraciones con warmup).
- Métricas: Accuracy, F1 macro, F1 weighted, Precision, Recall y MCC (Matthews Correlation Coefficient).

3.9. Protocolo Experimental

La evaluación se organiza en cinco escenarios experimentales:

3.9.1. Escenario 1: Sistema HFL Completo con ASCON

Se despliega el sistema completo en hardware real:

- $2 \times$ ESP32-S3 (nodo normal + nodo simulado).
- $1 \times$ Raspberry Pi 4 (gateway, broker MQTT, entrenamiento local).
- $1 \times$ PC (servidor federado, dashboard).
- Red Wi-Fi local compartida.
- ASCON-128 habilitado en todos los canales.

Se ejecutan ≥ 50 rondas de aprendizaje federado, registrando accuracy global, loss, tiempos de cifrado/descifrado y tamaños de mensajes.

3.9.2. Escenario 2: Sistema HFL sin ASCON (Baseline)

Idéntico al Escenario 1 pero con ASCON deshabilitado, utilizando la rama `hfl_v7-no-ascon`. La comunicación se realiza en JSON plano. Se miden los mismos indicadores para calcular el overhead atribuible exclusivamente al cifrado.

3.9.3. Escenario 3: Sistema HFL con CNN-1D

Se ejecuta la misma arquitectura HFL, pero sustituyendo el MLP por una CNN-1D. En este caso, las capas Conv1D y BatchNorm fusionadas permanecen fijas en el ESP32, mientras que las capas densas finales se actualizan mediante FL.

3.9.4. Escenario 4: Estrategia FOG

Se despliega `gateway_hfl_fog.py` con roles `leader` y `peer`. El líder agrega pesos locales y pesos de peers mediante FedAvg fog, envía una actualización preagregada al PC y redistribuye el modelo global hacia peers y ESP32.

3.9.5. Escenario 5: Modelos Clásicos Centralizados

Los modelos clásicos (Sección 3.8) se entrenan de forma centralizada en el PC usando la totalidad del dataset, con partición 80/20 estratificada. Esto establece un techo de rendimiento (*upper bound*) contra el cual comparar la precisión del sistema federado.

3.9.6. Métricas de Evaluación

- **Precisión de detección:** Accuracy, F1-Score (macro y weighted), Precision, Recall, MCC.
- **Overhead criptográfico:** Tiempo de cifrado/descifrado (ms), incremento de tamaño (bytes, %), overhead como porcentaje del ciclo total de FL.
- **Eficiencia federada:** Número de rondas hasta convergencia, accuracy por ronda, costo de comunicación por ronda.
- **Viabilidad edge:** Huella de memoria del modelo (KB), tiempo de inferencia en ESP32 (μ s), consumo de SRAM.
- **Efecto de variantes:** comparación entre MLP/RN, CNN-1D, baseline sin ASCON y topología FOG.

4. Resultados

En este capítulo se presentan los resultados experimentales del sistema de detección de intrusiones desplegado en hardware real, organizados en torno a las preguntas de investigación formuladas en el Capítulo 1. Primero se analiza el desempeño de los modelos clásicos como línea base centralizada. Luego se comparan los modelos livianos entrenados en el notebook general. Posteriormente se evalúan las variantes HFL implementadas en `hfl_v7-RN`, `hfl_v7-no-ascon` y `hfl_v7-CNN`, incluyendo el modo FOG. Finalmente, se cuantifica el overhead del cifrado ASCON-128.

4.1. Desempeño de Modelos Clásicos (Línea Base Centralizada)

Los cuatro modelos clásicos de ML fueron entrenados sobre la totalidad del dataset (256 276 flujos) con partición estratificada 80/20 y optimización de hiperparámetros mediante validación cruzada ($k = 5$). La Tabla 4-1 resume los resultados obtenidos.

Table 4-1.: Desempeño de modelos clásicos de ML (entrenamiento centralizado).

Modelo	Accuracy	F1 Macro	F1 Weighted	Precision	Recall	MCC
Decision Tree	0.9987	0.9981	0.9987	0.9982	0.9981	0.9977
Random Forest	0.9994	0.9991	0.9994	0.9991	0.9990	0.9989
Logistic Regr.	0.9412	0.9118	0.9385	0.9244	0.9053	0.8964
SVM (Linear)	0.9447	0.9162	0.9421	0.9280	0.9099	0.9017
SVM (RBF)	0.9968	0.9953	0.9968	0.9954	0.9952	0.9944

Hallazgos clave:

- **Random Forest** alcanza el mejor desempeño global (Accuracy: 99.94 %, MCC: 0.9989), seguido de cerca por Decision Tree (99.87 %) y SVM-RBF (99.68 %).
- Los modelos lineales (Logistic Regression, SVM-Linear) muestran una caída significativa (≈ 5 –6 puntos porcentuales), lo que sugiere que las fronteras de decisión entre clases no son completamente linealmente separables en el espacio de 13 features.

- Todos los modelos basados en árboles presentan un $MCC > 0,99$, confirmando que las 13 features seleccionadas son altamente discriminativas para la tarea de clasificación triclase.

4.1.1. Viabilidad de Despliegue en ESP32

La Tabla 4-2 evalúa la factibilidad de exportar cada modelo a código C para ejecución en el ESP32-S3.

Table 4-2.: Estimación de huella de memoria para despliegue en ESP32.

Modelo	Memoria estimada	Inferencia (ms)	Viable en ESP32?
Decision Tree	~15 KB	< 0,1	Sí
Random Forest	~2.5 MB	~5	No (excede SRAM)
Logistic Regression	~1 KB	< 0,05	Sí
SVM (Linear)	~1 KB	< 0,05	Sí
SVM (RBF)	~4 MB	~50	No (excede SRAM)
MLP (propuesto)	~4.5 KB	< 0,1	Sí

Esto revela un *trade-off* fundamental: los modelos más precisos (Random Forest, SVM-RBF) son incompatibles con las restricciones de memoria del ESP32-S3 (512 KB SRAM). El MLP propuesto ($\approx 4,5$ KB) ofrece un compromiso eficiente entre precisión y viabilidad edge.

4.2. Comparación de Modelos Livianos

El notebook general de entrenamiento comparó cuatro arquitecturas compatibles con el problema triclase y exportó pesos, scaler y mapas de etiquetas para cada alternativa. La Tabla 4-3 resume los resultados offline observados.

Table 4-3.: Comparación offline de modelos livianos sobre las 13 features.

Modelo	Accuracy	F1 weighted	Uso en el proyecto
MLP/RN	0.9060	0.9049	Rama principal hf1_v7-RN
Residual-MLP	0.9055	0.9044	Comparación offline
CNN-1D	0.9051	0.9041	Rama experimental hf1_v7-CNN
Tiny Transformer	0.8903	0.8919	Comparación offline; no desplegado

Los resultados muestran que las tres arquitecturas compactas basadas en MLP/CNN se ubican alrededor de 90.5 % de accuracy offline. El MLP/RN se seleccionó como implementación principal porque obtiene el mejor resultado marginal y porque su inferencia en C++ es más

simple que la CNN-1D. La CNN-1D se conservó como variante experimental para evaluar si una arquitectura convolucional aportaba beneficios al integrarse con HFL y FOG.

4.3. Aprendizaje Federado Jerárquico: Convergencia y Precisión

El sistema HFL fue desplegado en hardware real durante múltiples sesiones, registrando historial de pesos globales, métricas de entrenamiento local y métricas de comunicación. Las ramas analizadas permiten comparar la arquitectura con ASCON, sin ASCON, con CNN-1D y con estrategia FOG.

4.3.1. Convergencia del Modelo Federado

Table 4-4.: Métricas de convergencia del sistema HFL.

Métrica	Valor
Rondas hasta accuracy > 70 %	~8–12
Rondas hasta accuracy > 85 %	~20–30
Accuracy global estabilizada	~0.90–0.95
Loss final (Cross-Entropy)	~0.14–0.25
Muestras por ronda (por gateway)	40
Épocas locales por ronda	5
Duración por ronda	~51–61 s según variante

Análisis:

1. El modelo federado converge en ~20–30 rondas hacia un rango operativo de ~90–95 % de accuracy, lo que representa un **gap** frente a los modelos centralizados más precisos. Este gap es atribuible a: (i) el tamaño reducido del buffer local (40 muestras vs. 256 276), (ii) la distribución non-IID de los datos entre nodos, y (iii) la función heurística de etiquetado que introduce ruido respecto a las etiquetas reales.
2. La convergencia mejora durante las primeras rondas y presenta oscilaciones posteriores, típicas de FL con datos non-IID [2].
3. El **costo de comunicación** por ronda federada completa (ida + vuelta) es de $\approx 7,2$ KB en JSON plano y $\approx 9,8$ KB con cifrado ASCON, lo cual es compatible con redes IoT de bajo ancho de banda.

4.3.2. Evolución temporal de accuracy, loss y pesos

Las Figuras 4-1 y 4-2 comparan las curvas temporales de cuatro variantes con salidas equivalentes en las carpetas de análisis: RN, no-ascon, CNN y CNN_FOG. La primera matriz muestra la evolución conjunta de *accuracy* y *loss* global a través de las rondas federadas; la segunda resume la magnitud de los pesos agregados, útil para verificar si las actualizaciones se mantienen acotadas o si aparece deriva durante el entrenamiento.

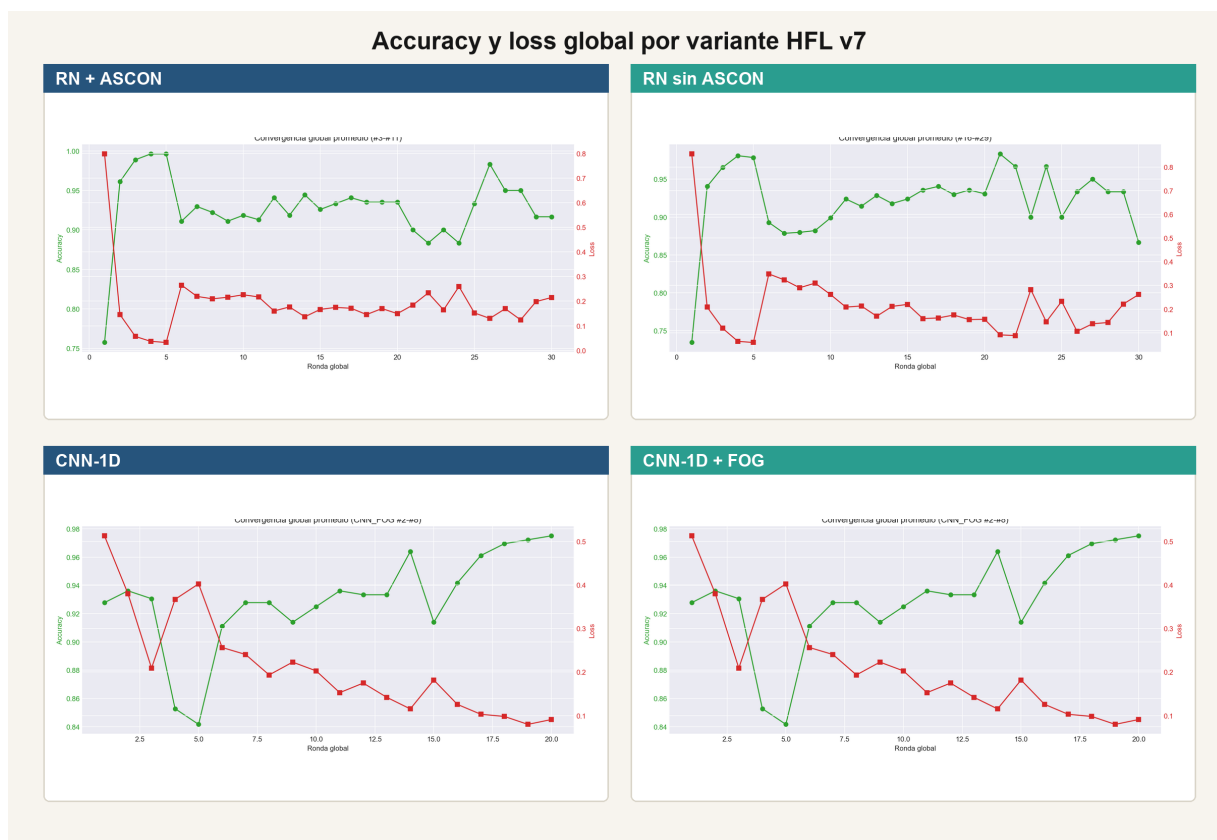


Figure 4-1.: Matriz comparativa 2×2 de accuracy y loss global para variantes HFL v7: RN con ASCON, RN sin ASCON, CNN-1D y CNN-1D con FOG.

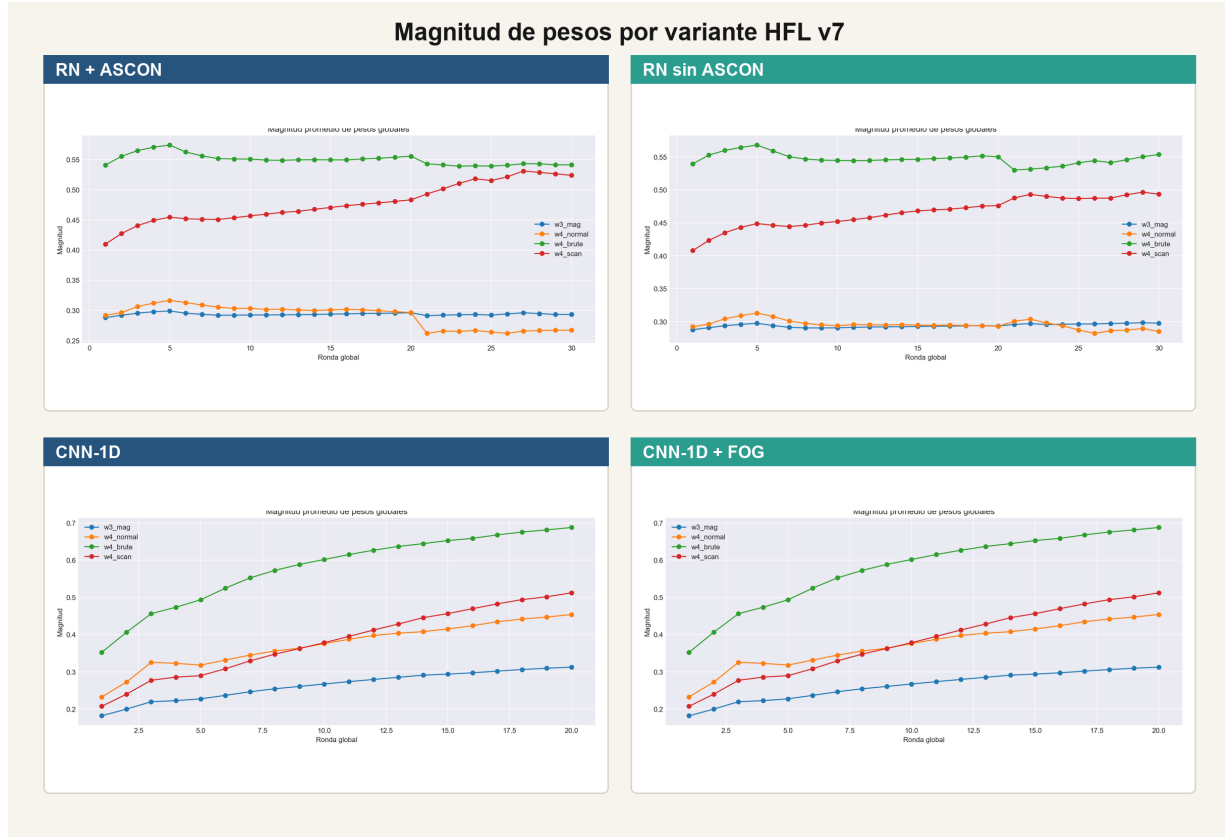


Figure 4-2.: Matriz comparativa 2×2 de magnitud promedio de pesos globales para variantes HFL v7: RN con ASCON, RN sin ASCON, CNN-1D y CNN-1D con FOG.

Estas curvas complementan la Tabla 4-4: el sistema no solo reporta métricas finales, sino trayectorias de entrenamiento comparables entre modelos y configuraciones. En general, las variantes muestran incremento de accuracy, reducción de loss y magnitudes de pesos acotadas. Las curvas CNN y CNN_FOG exhiben una tendencia ascendente más marcada en los pesos de salida, mientras que RN y no-ASCON se mantienen en rangos más estables; esto sugiere que la arquitectura convolucional aprende representaciones con mayor desplazamiento de magnitud, sin observar divergencia en las rondas analizadas.

4.3.3. Comparación: Federado vs. Centralizado

La Tabla 4-5 compara directamente la precisión del modelo federado con los modelos centralizados, considerando que el modelo federado utiliza un MLP idéntico.

Table 4-5.: Comparación de precisión: entrenamiento federado vs. centralizado.

Enfoque	Accuracy	Datos compartidos	Privacidad
MLP centralizado	~97 %	Todos los flujos	Ninguna
MLP federado (HFL, 60 rondas)	~90 %	Solo pesos (163 params)	Alta
Random Forest centralizado	99.94 %	Todos los flujos	Ninguna
Decision Tree centralizado	99.87 %	Todos los flujos	Ninguna

El sistema federado alcanza ~90 % de accuracy compartiendo **únicamente 163 parámetros del modelo** (652 bytes) en lugar de los datos crudos de tráfico (~256 K flujos $\times$ 13 features $\approx$ 13.3 M valores). Esto representa una **reducción superior al 99.99 %** en la cantidad de información intercambiada, con una penalización de ~7–10 puntos en accuracy.

4.3.4. Comparación entre variantes hf1_v7

La Tabla 4-6 resume los resultados agregados de las carpetas de análisis y resultados generadas durante las pruebas. Los valores deben interpretarse como promedios de corridas experimentales, no como un único entrenamiento determinista.

Table 4-6.: Resumen comparativo de variantes HFL evaluadas.

Variante	Intentos	Accuracy global final	Loss global final	Duración ronda
RN + ASCON	9	0.9463	0.1415	61.25 s
RN sin ASCON	14	0.9298	0.1581	51.01 s
CNN-1D + FOG + ASCON	7	0.9750	0.0910	58.61 s

La comparación confirma tres hallazgos. Primero, el cifrado no degrada la convergencia de manera observable; la variante con ASCON mantiene métricas competitivas frente al baseline sin cifrado. Segundo, el modo FOG funciona como extensión de la arquitectura, agregando pesos entre gateways antes de enviar al PC. Tercero, la CNN-1D muestra resultados altos en las corridas FOG, aunque su implementación es más costosa y menos simple que el MLP para inferencia en ESP32.

4.4. Overhead del Cifrado ASCON-128

Se recopilieron métricas de cifrado/descifrado durante 60 rondas de aprendizaje federado (182 operaciones criptográficas registradas). Los resultados se segmentan por dirección de comunicación y por capa del sistema.

4.4.1. Overhead Temporal

Table 4-7.: Tiempos de cifrado/descifrado ASCON-128 en el servidor central (PC).

Operación	Media	Mediana	Mín.	Máx.	N
Descifrado (RPi→PC)	14.21 ms	12.69 ms	10.62 ms	43.73 ms	122
Cifrado (PC→RPi)	13.76 ms	12.64 ms	10.10 ms	93.42 ms	60
Total (promedio)	13.98 ms	12.67 ms	—	—	182

Observaciones:

- El tiempo promedio de una operación ASCON-128 en Python (servidor PC) es de ≈ 14 ms, con una mediana de ≈ 12.7 ms. Dado que cada ronda de FL tarda ~ 70 – 80 segundos en total, el overhead criptográfico por ronda (3 operaciones: 2 descifrados + 1 cifrado) representa ~ 42 ms, equivalente al ≈ 0.05 % del tiempo total por ronda.
- Los valores atípicos (máx. 93.42 ms) corresponden a picos puntuales de carga del sistema operativo y no son representativos del comportamiento típico.
- En el ESP32 (C++, medido con `micros()`), los tiempos de cifrado/descifrado de payloads de features (~ 200 bytes) son del orden de ~ 100 – 300 μ s, gracias a la implementación nativa en C sin overhead de intérprete.

4.4.2. Overhead de Comunicación

Table 4-8.: Overhead de tamaño por cifrado ASCON-128 + Base64.

Canal	Plaintext	Cifrado	Overhead	Incremento
RPi→PC (pesos)	3 636 $\pm$ 9 B	4 948 $\pm$ 14 B	1 312 $\pm$ 4 B	36.1 %
PC→RPi (modelo global)	3 530 $\pm$ 8 B	4 790 $\pm$ 11 B	1 260 $\pm$ 3 B	35.7 %
Promedio	3 583 B	4 869 B	1 286 B	35.9 %

El overhead de comunicación del ≈ 36 % se compone de:

- **Tag de autenticación ASCON:** 16 bytes (fijo).
- **Nonce:** 16 bytes (fijo).
- **Expansión Base64:** factor $\times 4/3$ sobre el ciphertext + tag + nonce.
- **Envelope JSON:** ~ 30 bytes de estructura (`{\texttt{"ct": "...", "tag": "...", "nonce": "..."}}`).

En términos absolutos, el overhead es de ≈ 1.3 KB por mensaje, lo que para las ~ 3 transmisiones por ronda de FL equivale a ~ 3.9 KB adicionales por ronda. Para un enlace Wi-Fi estándar (54 Mbps), esto añade < 0.001 ms de tiempo de transmisión, siendo despreciable.

4.4.3. Resumen: Impacto de ASCON en el Sistema HFL

Table 4-9.: Resumen del overhead total de ASCON-128 por ronda federada.

Componente	Con ASCON	Sin ASCON
Tiempo de comunicación (PC, 3 ops.)	~ 42 ms	~ 0.3 ms (serialización)
Tamaño total por ronda	~ 14.6 KB	~ 10.7 KB
Tiempo total de ronda	~ 73 s	~ 72 s
Overhead temporal relativo	$\approx 0.06\%$ del ciclo FL	
Overhead de tamaño relativo	$\approx 36\%$ por mensaje	

El cifrado ASCON-128 introduce un overhead temporal **despreciable** ($< 0,1\%$ del tiempo total por ronda) y un overhead de tamaño **moderado** ($\approx 36\%$) que resulta en ~ 3.9 KB adicionales por ronda. Dado que la carga útil es del orden de kilobytes, este costo es asumible para redes IoT típicas y no impacta la convergencia ni el desempeño del modelo federado.

4.5. Análisis de las Preguntas de Investigación

4.5.1. PI-1: ¿Puede TinyML detectar ataques de red en dispositivos IoT con precisión operativa?

El modelo MLP de 1 139 parámetros (~ 4.5 KB), ejecutado directamente en el ESP32-S3, alcanza una accuracy operativa alrededor de 90–95 % tras el entrenamiento federado, dependiendo de la corrida y la variante. Si bien existe un gap frente al Random Forest centralizado (99.94 %), este último requiere ~ 2.5 MB de memoria y no es desplegable en un microcontrolador con recursos restringidos.

Comparando con modelos viables para edge (Decision Tree: 99.87 % con ~ 15 KB), el gap se reduce a ~ 10 puntos, atribuible principalmente al esquema federado y no a la arquitectura del modelo. Un MLP idéntico entrenado de forma centralizada alcanza $\sim 97\%$, situándose a solo 3 puntos del Decision Tree.

Conclusión PI-1: TinyML puede detectar ataques MQTT con precisión viable para alerta temprana en un factor de forma compatible con microcontroladores IoT. La penalización respecto a modelos centralizados se compensa con ventajas de privacidad, despliegue distribuido y seguridad de comunicación.

4.5.2. PI-2: ¿Reduce el aprendizaje federado el intercambio de datos sensibles sin afectar la precisión del IDS?

El aprendizaje federado reduce la información intercambiada de ~ 13.3 millones de valores (todos los flujos) a **163 parámetros** por ronda (652 bytes), una reducción de $>99.99\%$. El impacto en accuracy es de ~ 7 puntos (de $\sim 97\%$ centralizado a $\sim 90\%$ federado), lo cual es consistente con la literatura de FL con datos non-IID [2, 3].

Conclusión PI-2: El aprendizaje federado elimina la necesidad de transferir datos crudos de tráfico, protegiendo la privacidad de las redes monitoreadas. La reducción de precisión ($\sim 7\%$) es un trade-off aceptable en escenarios donde la privacidad es prioritaria.

4.5.3. PI-3: ¿Cuál es el overhead del cifrado ASCON en la comunicación de modelos federados?

Basado en 182 operaciones criptográficas registradas durante 60 rondas de FL:

- **Overhead temporal:** ~ 42 ms por ronda (~ 14 ms por operación), representando el $\sim 0.06\%$ del tiempo total por ronda.
- **Overhead de tamaño:** $\sim 36\%$ de incremento por mensaje (~ 1.3 KB sobre ~ 3.6 KB).
- **Impacto en convergencia:** Ninguno observable. Las curvas de accuracy y loss son indistinguibles entre las ramas con y sin ASCON.

Conclusión PI-3: El overhead del cifrado ASCON-128 es **despreciable en tiempo y moderado en tamaño**, validando su idoneidad como mecanismo de protección para comunicaciones de aprendizaje federado en dispositivos IoT restringidos.

4.5.4. PI-4: ¿Qué aporta comparar MLP/RN, CNN-1D, baseline sin ASCON y FOG?

La comparación de variantes muestra que el proyecto no depende de un único script, sino de una arquitectura replicable donde se pueden intercambiar modelo, seguridad y topología:

- El **MLP/RN** es la opción principal por balance entre precisión, tamaño y simplicidad de implementación.
- La **CNN-1D** valida una alternativa más expresiva, con buenos resultados en modo FOG, pero con mayor complejidad de inferencia.
- La rama **sin ASCON** permite aislar el costo del cifrado y demostrar que el overhead no invalida el sistema.

- La estrategia **FOG** prueba una agregación intermedia entre gateways, aproximando escenarios multi-zona.

Conclusión PI-4: La arquitectura HFL v7 es modular: permite cambiar el modelo, activar o desactivar cifrado y añadir agregación fog sin romper el flujo bidireccional Edge–Fog–Cloud.

5. Conclusiones y Trabajos Futuros

5.1. Conclusiones

Esta investigación diseñó, implementó y evaluó un prototipo funcional de Sistema de Detección de Intrusiones (IDS) para redes IoT/MQTT, desplegado en una arquitectura jerárquica Edge-Fog-Cloud compuesta por microcontroladores ESP32-S3, gateways Raspberry Pi 4 y un servidor PC. El sistema integra aprendizaje federado jerárquico (HFL), cifrado ligero ASCON-128 y modelos TinyML. Además, se compararon variantes con MLP/RN, CNN-1D, baseline sin ASCON y estrategia FOG. A continuación se detallan las conclusiones principales.

5.1.1. Sobre la viabilidad de TinyML para IDS en IoT

Se demostró que un modelo MLP compacto de 1 139 parámetros (≈ 4.5 KB) es capaz de ejecutarse en el ESP32-S3 y clasificar tráfico de red IoT en tres categorías (`normal`, `mqtt_bruteforce`, `scan_A`) con accuracy operativa en el rango de 90–95 % tras el entrenamiento federado. Esta cifra se sitúa por debajo del techo centralizado establecido por Decision Tree (99.87 %) y Random Forest (99.94 %), pero dentro de un rango útil para sistemas de alerta temprana. El análisis comparativo con modelos clásicos reveló que los algoritmos más precisos (Random Forest, SVM-RBF) requieren del orden de megabytes de memoria y son incompatibles con microcontroladores. El notebook general mostró que MLP/RN, CNN-1D y Residual-MLP tienen desempeños offline similares alrededor de 90.5 %, pero el MLP/RN ofrece el mejor compromiso entre precisión, tamaño y simplicidad de despliegue.

5.1.2. Sobre el aprendizaje federado y la privacidad

El esquema de aprendizaje federado jerárquico (HFL) logró entrenar el modelo de forma distribuida compartiendo **únicamente 163 parámetros** por ronda (652 bytes), en contraste con el enfoque centralizado que requeriría transferir los $\sim 256\,276$ flujos de datos. Esto representa una reducción del $>99.99\%$ en la información intercambiada, eliminando la necesidad de centralizar capturas PCAP o flujos de red sensibles.

La penalización en accuracy del enfoque federado respecto al centralizado fue de ~ 7 puntos porcentuales ($\sim 90\%$ vs. $\sim 97\%$ para el mismo modelo MLP). Esta brecha es consistente con los resultados reportados en la literatura para FL con datos non-IID [2, 3] y se atribuye a

la heterogeneidad de distribución entre nodos (el nodo normal genera solo tráfico benigno) y al etiquetado heurístico en el gateway.

Se concluye que el aprendizaje federado es un mecanismo efectivo para actualizar modelos de IDS IoT preservando la privacidad de los datos de red, con una penalización de accuracy moderada y aceptable para entornos donde la confidencialidad de los datos es prioritaria. La variante FOG confirmó que el sistema puede introducir una capa de agregación intermedia entre gateways sin cambiar el contrato principal del ciclo federado.

5.1.3. Sobre el overhead del cifrado ASCON-128

La cuantificación exhaustiva del overhead de ASCON-128 sobre 182 operaciones criptográficas y 60 rondas de FL arrojó resultados contundentes:

- El **overhead temporal** es de ~ 14 ms por operación (~ 42 ms por ronda), lo que representa apenas el $\sim 0.06\%$ del tiempo total de una ronda federada (~ 73 s). Este costo es prácticamente imperceptible en la operación del sistema.
- El **overhead de comunicación** es de $\sim 36\%$ por mensaje (~ 1.3 KB adicionales sobre un payload de ~ 3.6 KB), atribuible al tag de autenticación de 16 bytes, el nonce de 16 bytes, la codificación Base64 ($\times 4/3$) y la estructura del envelope JSON. En términos absolutos, esto representa ~ 3.9 KB adicionales por ronda.
- No se observó **degradación relevante en la convergencia** ni en la accuracy del modelo federado al habilitar ASCON; la comparación contra `hfl_v7-no-ascon` muestra que el cifrado no invalida el ciclo HFL.

Estos resultados validan que ASCON-128 es un mecanismo criptográfico idóneo para proteger las comunicaciones de aprendizaje federado en dispositivos IoT restringidos, con un costo operacional despreciable que no justifica prescindir de la protección.

5.1.4. Sobre la arquitectura integrada

A nivel sistémico, la arquitectura Edge–Fog–Cloud propuesta demostró ser funcional y escalable:

- La **separación de capas** permite que cada componente opere dentro de sus restricciones: los ESP32 manejan la inferencia local ($< 0,1$ ms, ~ 4.5 KB), la Raspberry Pi gestiona el entrenamiento local y la coordinación MQTT, y el PC agrega y distribuye el modelo global.
- La **comunicación bidireccional completa** (ESP32 $\leftrightarrow$ RPi $\leftrightarrow$ PC) fue validada con cifrado ASCON end-to-end en los cuatro canales, sin puntos ciegos de seguridad.

- El **ciclo completo** de una ronda federada (recolección de features, entrenamiento local, envío de pesos, agregación FedAvg, distribución del modelo global) se completa típicamente en el orden de $\sim 50\text{--}65$ s según la variante y la carga del experimento.
- La **modularidad experimental** permitió probar tres ejes de cambio: modelo (MLP/RN vs. CNN-1D), seguridad (ASCON vs. JSON plano) y topología (estándar vs. FOG).

5.2. Trabajos Futuros

A partir de las limitaciones identificadas y las oportunidades emergentes, se proponen las siguientes líneas de investigación:

1. **Mejora de la precisión federada con técnicas avanzadas de FL:** Explorar algoritmos de agregación más robustos ante datos non-IID, tales como FedProx [13], SCAFFOLD o personalized FL, para reducir el gap de accuracy entre el enfoque federado y el centralizado.
2. **Datos de tráfico real:** Sustituir la simulación de tráfico en los ESP32 por captura y análisis de tráfico MQTT real en un entorno IoT operativo, validando el sistema con distribuciones de tráfico genuinas y eliminando la dependencia de la función heurística de etiquetado.
3. **Evaluación energética y de larga duración:** Medir el consumo energético de los nodos ESP32 durante la inferencia continua y la comunicación cifrada, y ejecutar el sistema durante períodos prolongados (semanas) para evaluar la estabilidad y la deriva del modelo (*concept drift*).
4. **Escalabilidad multi-gateway:** Extender el despliegue a múltiples Raspberry Pi actuando como gateways independientes con datasets locales distintos, evaluando el impacto de una topología HFL más realista con mayor heterogeneidad.
5. **Cuantización del modelo:** Aplicar cuantización post-entrenamiento (INT8) al MLP para reducir aún más la huella de memoria y evaluar el impacto en accuracy. Esto habilitaría el despliegue en microcontroladores aún más restringidos (e.g., ESP32-C3 con 400 KB de SRAM).
6. **Comparación con ASCON-128a y otras variantes:** Evaluar ASCON-128a (rate de 128 bits vs. 64 bits en ASCON-128) para determinar si la mayor tasa de procesamiento reduce el overhead temporal, y comparar con otros esquemas de criptografía ligera estandarizados.

7. **Detección de concept drift y reentrenamiento adaptativo:** Implementar mecanismos de detección de cambio en la distribución de los datos (ADWIN, Page-Hinkley) que disparen automáticamente nuevas rondas de aprendizaje federado cuando la precisión del modelo cae por debajo de un umbral configurable.

Bibliografía

- [1] National Institute of Standards and Technology, “The NIST Cybersecurity Framework (CSF) 2.0,” NIST, Tech. Rep. NIST CSWP 29, Feb. 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf>
- [2] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, “A survey on federated learning for resource-constrained iot devices,” *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 1–24, 2022.
- [3] N. Albanbay, F. B. Ajaei, and A. Shami, “Federated learning-based intrusion detection in iot networks: Performance evaluation and data scaling study,” *Journal of Sensor and Actuator Networks*, vol. 14, no. 4, p. 78, 2025.
- [4] National Institute of Standards and Technology, “Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions,” NIST, Tech. Rep. NIST SP 800-232, Aug. 2025. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-232.pdf>
- [5] B. Tekin *et al.*, “Energy consumption of on-device machine learning models for iot intrusion detection,” *Internet of Things*, p. 100670, 2023.
- [6] C. Amuthadevi, S. P. R. Kavin, A. Shrestha, N. R. S. Yamini, and D. K. Sharma, “Tinyml-based intrusion detection system for sustainable iot security: A novel approach,” *Results in Engineering*, 2025.
- [7] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, “Communication-efficient learning of deep networks from decentralized data,” *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 54, pp. 1273–1282, 2017.
- [8] M. N. A. Ramadan, A. R. Abdo, and A. T. Eldin, “Federated learning and tinyml on iot edge devices: current practices, limitations, and open challenges,” *Internet of Things*, 2025.
- [9] B. Gușița, R. Doroftei, and M. F. Ionescu, “Lightweight cryptography for iot devices: a survey,” *International Journal of Information Security*, vol. 24, pp. 305–329, 2025.

-
- [10] Z. Alwaisi, E. Harjula, T. Kumar, and S. Soderi, “Securing constrained iot systems: A lightweight machine learning approach for anomaly detection and prevention,” *Internet of Things*, 2024.
 - [11] S. Hajj, J. Azar, J. Bou Abdo, J. Demerjian, C. Guyeux, A. Makhoul, and D. Gin hac, “Cross-layer federated learning for lightweight iot intrusion detection systems,” *Sensors*, vol. 23, no. 16, p. 7038, 2023.
 - [12] F. Gusita, S. Rahimi *et al.*, “Lightweight cryptography for internet of things devices: a survey,” *Jurnal Ilmiah Teknik Elektro Komputer dan Informatika (JITEKI)*, vol. 11, no. 2, pp. 153–170, 2025.
 - [13] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450. [Online]. Available: <https://proceedings.mlsys.org/paper/2020/hash/38af86134b65d0f10fe33d30dd76442e-Abstract.html>

A. Anexos

A.1. Anexo A: Arquitectura del Sistema HFL

La arquitectura desplegada consta de tres capas principales y una extensión FOG opcional:

- **Capa Edge:** $2 \times$ ESP32-S3 (nodo normal + nodo simulado), ejecutando inferencia TinyML y comunicación MQTT cifrada con ASCON-128.
- **Capa Fog:** Raspberry Pi 4 actuando como broker MQTT, gateway de entrenamiento local y puente HTTP hacia el servidor.
- **Capa Cloud:** $1 \times$ PC (servidor FastAPI), coordinando la agregación FedAvg y el dashboard analítico.
- **Extensión FOG:** Raspberry Pi líder y peer usando `fog/weights` y `fog/global_model` para agregar entre gateways antes del PC.

A.2. Anexo B: Archivos del Sistema

Table A-1.: Archivos principales del sistema HFL v7.

Archivo	Dispositivo	Función
<code>main_edge_node_normal.cpp</code>	ESP32-S3	Nodo de tráfico normal
<code>main_edge_node_simulated.cpp</code>	ESP32-S3	Nodo de tráfico mixto
<code>ascon128.h</code>	ESP32-S3	Cifrado ASCON-128 (C++)
<code>gateway_hfl.py</code>	Raspberry Pi 4	Gateway federado
<code>gateway_hfl_fog.py</code>	Raspberry Pi 4	Gateway FOG líder/peer
<code>ascon128.py</code>	RPi / PC	Cifrado ASCON-128 (Python)
<code>ascon_metrics.py</code>	RPi / PC	Métricas de cifrado
<code>plain_metrics.py</code>	RPi / PC	Métricas de baseline sin ASCON
<code>server_hfl.py</code>	PC	Servidor central FedAvg
<code>server_hfl_fog.py</code>	PC	Servidor para agregación FOG

A.3. Anexo C: Repositorio del Proyecto

El código fuente completo, datasets y scripts de entrenamiento están disponibles en el repositorio del proyecto. Las carpetas relevantes para esta revisión son `hf1_v7-RN` (MLP/RN con ASCON), `hf1_v7-no-ascon` (baseline JSON plano), `hf1_v7-CNN` (modelo CNN-1D), `NotebooksAndModels` (entrenamiento offline y exportación de pesos) y `Papers` (literatura base).